

dbt on Snowflake — Data Engineering Guide

The T in ELT: models, tests, snapshots, incremental — with interview prep

[ref/source](#) · [materializations](#) · [incremental/MERGE](#) · [snapshots \(SCD2\)](#) · [tests](#) · [Jinja/macros](#) · [deploy](#)

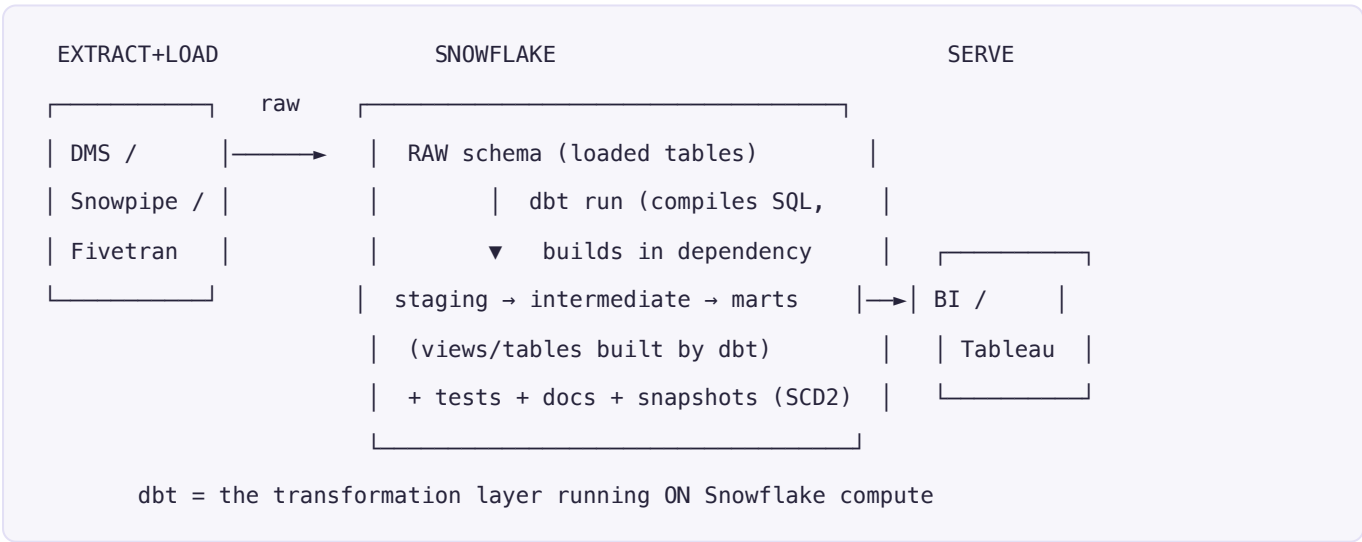
dbt (data build tool) is how modern data teams do the **"T" in ELT** — transforming raw data *inside* the warehouse with version-controlled SQL, tests, and documentation. This guide shows how dbt works **on Snowflake**, end to end, using our running example: **"RetailCo."**

Prereq: you've loaded raw data into Snowflake (via S3 stages / COPY / Snowpipe — see the Snowflake guide). dbt takes over from there and builds your clean, tested, modeled tables.

PART 0 — What dbt is (and what it is not)

dbt is a **transformation framework**. You write **SELECT statements** (models); dbt turns them into **CREATE TABLE / CREATE VIEW** in Snowflake, in the **right order**, and can **test** and **document** them. It brings software-engineering practices to analytics SQL:

- **It runs SQL in Snowflake** — dbt itself does no data processing; Snowflake's warehouse does the compute. dbt is the orchestrator/compiler.
- **It's the T in ELT** — Extract & Load happen first (Fivetran/Airbyte/DMS/Snowpipe land raw data), then **dbt Transforms** it in-warehouse.
- **What it gives you:** dependency management (a DAG via `ref()`), version control (Git), **tests** (data quality), **documentation + lineage**, reusable logic (**Jinja macros**), and environments (dev/prod).
- **dbt Core** = free open-source CLI. **dbt Cloud** = managed service adding a scheduler, browser IDE, CI, and hosted docs.



PART 1 — How dbt connects to Snowflake

dbt connects using a **profile** (`profiles.yml`). It tells dbt which Snowflake account, role, warehouse, database, and schema to build into.

```
# ~/.dbt/profiles.yml
retailco:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "ab12345.us-east-1"      # your Snowflake account locator
      user: "PRAMOD"
      authenticator: username_password_mfa  # or key-pair / SSO (prod: key-pair)
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"
      role: "TRANSFORMER"               # a dbt role with rights to build
      warehouse: "TRANSFORM_WH"        # dbt's compute
      database: "RETAILCO"
      schema: "DBT_PRAMOD"              # your dev schema (isolated)
      threads: 8                        # parallel model builds
```

Best practice: a dedicated **TRANSFORMER role** and **TRANSFORM_WH** warehouse for dbt, key-pair auth in production, and per-developer dev schemas.

PART 2 — Project structure

```
retailco_dbt/
├─ dbt_project.yml          # project config (name, model paths, materializations)
├─ models/
│   ├─ staging/             # 1:1 cleanups of raw sources (views)
│   │   ├─ _sources.yml     # declares raw tables + tests + freshness
│   │   ├─ stg_orders.sql
│   │   └─ stg_customers.sql
│   ├─ intermediate/        # reusable joins/logic (optional layer)
│   │   └─ int_orders_enriched.sql
│   └─ marts/               # business-facing tables (star schema)
│       ├─ fct_orders.sql
│       ├─ dim_customers.sql
│       └─ daily_revenue.sql
├─ snapshots/              # SCD Type 2 history
│   └─ customers_snapshot.sql
├─ seeds/                  # small CSVs loaded as tables (e.g., country codes)
├─ macros/                 # reusable Jinja SQL functions
├─ tests/                  # custom (singular) tests
└─ packages.yml            # external packages (e.g., dbt_utils)
```

```
# dbt_project.yml (key bits)
name: retailco
profile: retailco
models:
  retailco:
    staging:      { +materialized: view }      # staging = views (cheap, always fresh)
    intermediate: { +materialized: view }
    marts:        { +materialized: table }     # marts = tables (fast for BI)
```

PART 3 — Models, `ref()`, `source()` (the core idea)

A **model** is a `.sql` file containing one `SELECT`. dbt wraps it in DDL and builds it.

Sources = your raw tables (declared, not built by dbt):

```
# models/staging/_sources.yml
sources:
  - name: raw
    database: RETAILCO
    schema: RAW
    tables:
      - name: orders
        freshness:                # alert if raw data is stale
          warn_after: {count: 12, period: hour}
        columns:
          - name: order_id
            tests: [unique, not_null] # test the source directly
      - name: customers
```

Staging model — reference a source with `source()`:

```
-- models/staging/stg_orders.sql
select
  order_id,
  customer_id,
  amount::number(10,2)      as amount,
  lower(status)             as status,
  order_ts::timestamp_ntz   as order_ts
from {{ source('raw', 'orders') }}
where amount > 0            -- basic cleaning
```

Mart model — reference other models with `ref()` (this builds the **DAG**):

```
-- models/marts/fct_orders.sql
select
  o.order_id,
  o.customer_id,
  c.customer_name,
  o.amount,
  o.order_ts::date          as order_date
from {{ ref('stg_orders') }} o
join {{ ref('stg_customers') }} c using (customer_id)
```

Why `ref()` matters: dbt reads every `ref()` / `source()` to build a **dependency graph**, so `dbt run` builds models in the correct order and gives you **lineage** for free. You never hard-code table names — dbt resolves them per environment (dev vs prod schema).

Run it:

```
dbt run          # build all models
dbt run --select marts # build just the marts and what they depend on
```

PART 4 — Materializations (how a model is built)

The single most important dbt config. Set with `{{ config(materialized='...') }}` or in `dbt_project.yml`.

Materialization	What dbt does	Use when
view (default)	<code>CREATE VIEW</code>	staging; light logic; always-fresh, no storage
table	<code>CREATE TABLE AS</code> (full rebuild each run)	marts/BI; expensive logic reused often
incremental	first run builds table; later runs only add/merge new rows	large fact tables where full rebuild is too slow/costly
ephemeral	not built; inlined as a CTE into downstream models	small reusable logic you don't want as an object
snapshot	SCD Type 2 history table	tracking changes over time (see Part 6)

Incremental model (the big one for large facts)

```
-- models/marts/fct_events.sql
{{ config(materialized='incremental', unique_key='event_id') }}

select event_id, user_id, event_type, event_ts
from {{ source('raw', 'events') }}

{% if is_incremental() %}
    -- only new data since the max already loaded (dbt runs this filter on later runs)
    where event_ts > (select max(event_ts) from {{ this }})
{% endif %}
```

On Snowflake, dbt implements the incremental update with a **MERGE** using `unique_key`. This is how you load billions of rows without reprocessing history each run.

PART 5 — Tests (data quality — a huge dbt selling point)

dbt tests are assertions that **fail the pipeline** when data is wrong.

Generic tests (declared in YAML — the 4 built-ins):

```
# models/marts/_marts.yml
models:
  - name: fct_orders
    columns:
      - name: order_id
        tests: [unique, not_null]
      - name: status
        tests:
          - accepted_values: {values: ['completed', 'pending', 'cancelled']}
      - name: customer_id
        tests:
          - relationships: {to: ref('dim_customers'), field: customer_id} # FK integrity
```

Singular tests (custom SQL in `tests/` — the test passes if it returns **zero rows**):

```
-- tests/assert_no_negative_revenue.sql
select order_date, sum(amount) as revenue
from {{ ref('fct_orders') }}
group by order_date
having sum(amount) < 0
```

Run: `dbt test`. In CI, a failing test blocks the deploy → bad data never reaches BI.

PART 6 — Snapshots (SCD Type 2 history — common interview topic)

Sources usually show only the *current* value. A **snapshot** captures history so you can ask "what was this customer's tier last March?"

```
-- snapshots/customers_snapshot.sql
{% snapshot customers_snapshot %}
{{ config(
  target_schema='snapshots',
  unique_key='customer_id',
  strategy='timestamp',          -- or 'check' on specific columns
  updated_at='updated_at'
) }}
select * from {{ source('raw', 'customers') }}
{% endsnapshot %}
```

`dbt snapshot` adds `dbt_valid_from` / `dbt_valid_to` columns and inserts a new version each time a row changes — a true **SCD Type 2** table, maintained for you.

PART 7 — Jinja, macros & packages (DRY SQL)

dbt SQL is templated with **Jinja**, so you can write reusable logic:

```
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name) %}
    ({{ column_name }} / 100.0)::number(10,2)
{% endmacro %}
```

```
select order_id, {{ cents_to_dollars('amount_cents') }} as amount from ...
```

Packages add community macros — most teams install `dbt_utils` (helpers like `date_spine`, `surrogate_key`, generic tests) via `packages.yml` + `dbt deps`.

PART 8 — Documentation & lineage

```
dbt docs generate  # builds a docs site from your descriptions + compiled SQL
dbt docs serve     # opens it: searchable catalog + a visual DAG (lineage graph)
```

Add `description:` to models/columns in YAML and they appear in the docs. Lineage ("what feeds this table, what breaks if I change it") is auto-generated from `ref()`.

PART 9 — Running, building & deploying

```
dbt deps          # install packages
dbt seed          # load seed CSVs
dbt run           # build models
dbt test          # run tests
dbt snapshot      # update SCD2 snapshots
dbt build         # run seeds + models + tests + snapshots in DAG order (the everyday command)
```

Handy selectors: `dbt build --select stg_orders+` (a model and everything downstream), `--select state:modified+` (only what changed — for CI).

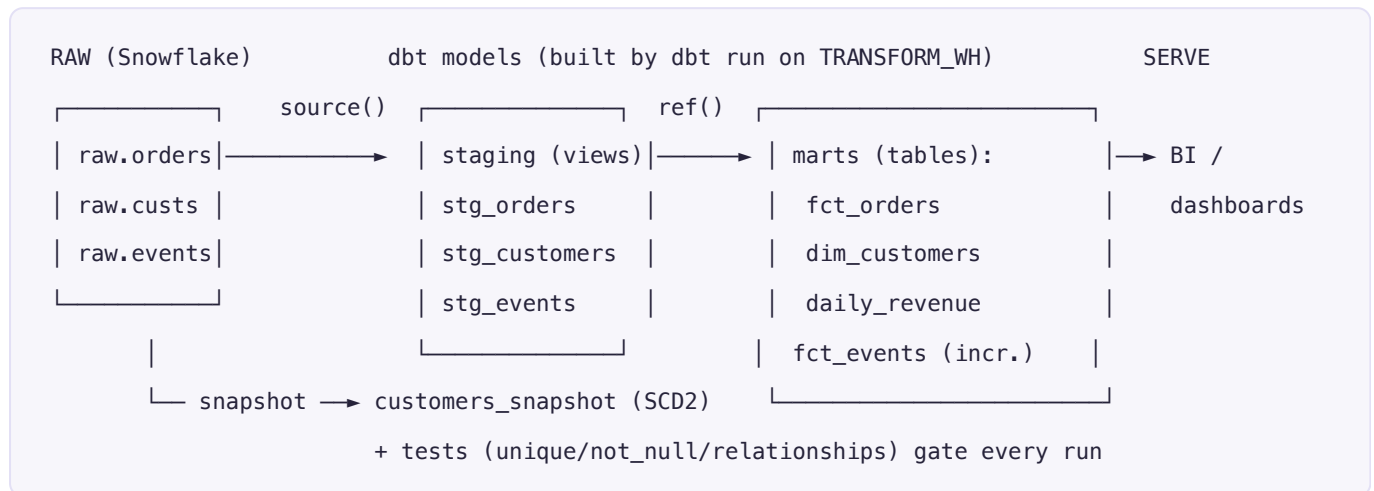
Deployment/orchestration:

- **dbt Cloud** — define **jobs** (`dbt build`) on a schedule, with CI on pull requests and hosted docs.
 - **dbt Core** — run in **Airflow/MWAA** (or `dbt-airflow`), Dagster, or GitHub Actions; containerized in CI/CD. Slim CI (`state:modified`) rebuilds only changed models.
-

THE SCENARIO — RetailCo: raw Snowflake → tested marts with dbt

Situation: raw `orders`, `customers`, `events` are already in `RETAILCO.RAW` (loaded by Snowpipe/COPY). Build trustworthy analytics with dbt.

Architecture



Steps:

1. **Declare sources** (`_sources.yml`) for the three raw tables + freshness + source tests.
2. **Staging** views (`stg_*`) clean/rename/type each source 1:1.
3. **Marts** (tables): `fct_orders` (join staging), `dim_customers`, `daily_revenue` (agg); `fct_events` as **incremental** (MERGE on `event_id`).
4. **Snapshot** `customers_snapshot` for SCD2 history.
5. **Tests** on keys/values/relationships; **singular test** for no-negative-revenue.
6. **dbt build** on a schedule (dbt Cloud job or Airflow) → BI reads the marts.
7. **CI:** on each PR, dbt Cloud/Slim CI builds only modified models in a temp schema and runs tests before merge.

Why it's a strong design: version-controlled SQL, a clear staging→marts layering, incremental for big facts, SCD2 via snapshots, automated data-quality tests that block bad deploys, and auto-generated lineage/docs — all running on Snowflake compute.

INTERVIEW Q&A

- **What is dbt and where does it fit?** The transformation (T) layer in ELT; it compiles and runs version-controlled SQL *in the warehouse* (Snowflake), managing dependencies, tests, and docs. It does no processing itself — Snowflake does.
- **ref() vs source() ?** `source()` points to raw tables dbt didn't build; `ref()` points to other dbt models. Both build the DAG and resolve names per environment.
- **Materializations?** view (default), table, incremental, ephemeral, snapshot — pick by cost/freshness/size (see Part 4).

- **How do incremental models work on Snowflake?** First run builds the table; later runs use `is_incremental()` to filter new rows and dbt issues a **MERGE** on `unique_key`.
- **How do you implement SCD Type 2?** dbt **snapshots** (`dbt_valid_from/to`), timestamp or check strategy.
- **How does dbt ensure data quality?** Generic tests (unique/not_null/accepted_values/relationships) + singular SQL tests; failing tests break the build (great in CI).
- **dbt Core vs Cloud?** Core = free CLI; Cloud adds scheduler, IDE, CI, hosted docs.
- **How do you orchestrate dbt?** dbt Cloud jobs, or Airflow/Dagster/GitHub Actions running `dbt build`; Slim CI (`state:modified+`) for fast PR checks.
- **What's `dbt build` vs `dbt run`?** `run` builds models; `build` runs seeds+models+tests+snapshots together in DAG order.
- **How does dbt handle environments?** Different targets/schemas in `profiles.yml` (dev per-developer schema, prod schema); same code, different destination.
- **Why dbt over stored procedures?** Version control, dependency graph, testing, docs/lineage, modularity (macros/packages), and CI/CD — software practices for analytics.
- **How does dbt relate to Snowflake features?** dbt = the T; it complements Snowflake's Streams/Tasks/Dynamic Tables. Many teams use dbt for modeling and Snowflake tasks/Airflow to schedule it; incremental models rely on Snowflake MERGE.

BEST PRACTICES & GOTCHAS

- **Layer models:** `staging` (views, 1:1 clean) → `intermediate` (reusable joins) → `marts` (business tables). One source table → one staging model.
- **Always use `ref()` / `source()`** — never hard-code table names.
- **Test keys and relationships;** add `source freshness` so stale raw data is caught.
- **Incremental for big facts; views for staging; tables for marts.**
- **Dedicated `TRANSFORMER` role + `TRANSFORM_WH`**; per-dev schemas; key-pair auth in prod.
- **Snapshots for history (SCD2); seeds for small static lookups.**
- **Use `dbt_utils`**; write macros for repeated logic; keep models small and readable.
- **CI with Slim CI (`state:modified+`)** to build only what changed.
- **Gotchas:** forgetting `is_incremental()` filter → full reload; wrong `unique_key` → duplicates on MERGE; snapshots must run regularly or you lose history between changes; ephemeral models can bloat compiled SQL if overused; a dev run into the wrong schema/target.